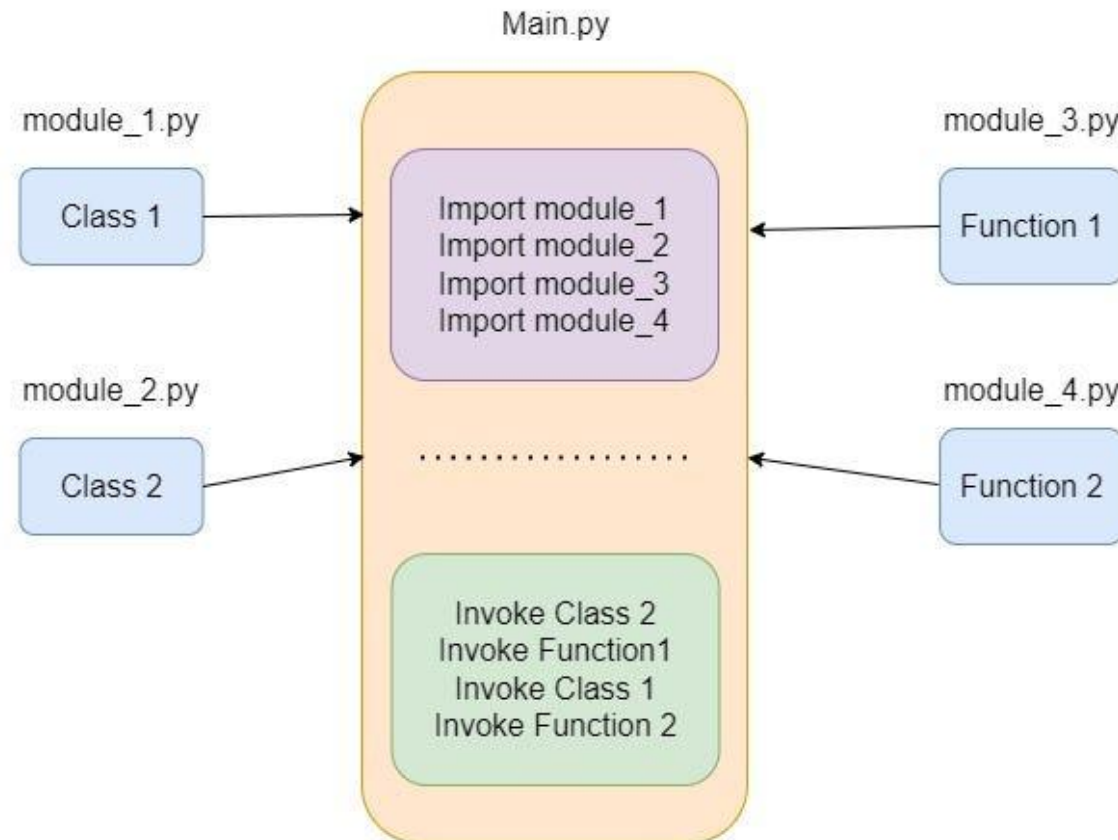


Modular programming is a method where a large program is broken down into distinct modules or files, each containing code related to a particular feature or function.



- Changing the program into separate pieces
- Code reusing
- Easier modification and maintenance of the program
- More understandable program
- Functions can be called from anywhere in the main program with different input data
- Functions enable the programmers to generate their own libraries of the functions.

- Python supports modular programming using modules (Python files containing related functions, classes, and variables)
- And packages (directories of modules).

An example of a reusable `geometry.py` module with geometric calculation

```
# geometry.py

PI = 3.14

def calculate_area(shape, *dimensions):
    if shape == "rectangle":
        return dimensions[0] * dimensions[1]
    elif shape == "circle":
        return PI * (dimensions[0] ** 2)
    else:
        return None

def calculate_perimeter(shape, *dimensions):
    if shape == "rectangle":
        return 2 * (dimensions[0] + dimensions[1])
    elif shape == "circle":
        return 2 * PI * dimensions[0]
    else:
        return None
```

By importing specific names, shapes.py gains access to the reusable functionalities provided by the module geometry.

```
# shapes.py

from geometry import PI, calculate_area, calculate_perimeter

rectangle_area = calculate_area("rectangle", 5, 3)
print(rectangle_area)

circle_perimeter = calculate_perimeter("circle", 8)
print(circle_perimeter)
```

Python Enhancement Protocol (PEP)

“A PEP is a design document providing information to the Python community, or describing a new feature for Python or its processes or environment.” (PEP 1)

Important Fundamentals

PEP 8: Style Guide for Python Code

PEP 20: The Zen of Python

PEP 257: Docstring Conventions

Bonus PEPs

PEP 484: Type Hints

PEP 498: Literal String Interpolation

PEP 572: Assignment Expressions

Your probably already familiar with...

- ▶ Using 4 spaces per indentation level (not tabs!)
- ▶ Putting two blank lines before functions and classes
- ▶ Limiting line lengths to:
 - ▶ 79 characters for code
 - ▶ 72 characters for long blocks of text
 - ▶ It is okay to increase the line length limit up to 99 characters

```
# !/usr/bin/env python3.7  
# -*- coding: UTF-8 -*-
```

```
"""This is a description of the module."""
```

```
import json  
import os
```

```
from astropy.table import Table, vstack
```

```
from my_code import utils
```

```
__version__ = 0.1
```

```
def my_function():  
    ...
```


PEP 8: Naming Convention

TYPE	NAMING CONVENTION	EXAMPLES
Function	Use lowercase words separated by underscores.	function, my_function
Variable	Use lowercase letters or word, or words separated with underscores. (I.e., snake_case)	x, var, my_variable
Class	Start each word with a capital letter. Do not separate words with underscores. (I.e., CamelCase)	Model, MyClass
Method	Use lowercase words separated with underscores.	class_method, method
Constant	Use an uppercase single letter, word, or words separated by underscores.	CONSTANT, MY_CONSTANT
Module	Use short lowercase words separated with underscores.	module.py, my_module.py
Package	Use short lowercase words without underscores.	package, mypackage

- ▶ Use ``is`` when comparing singletons
- ▶ Use ``is not`` instead of ``not ... is``

```
# Wrong  
if foo == None:  
    do_something()
```

```
# Also wrong  
if not foo is None:  
    do_something()
```

```
# Correct  
if foo is not None:  
    do_something()
```


PEP 8: General Recommendations

- ▶ Booleans are already Booleans - they don't need comparisons
- ▶ For sequences, (e.g., a lists), use the fact that empty sequences are false

Wrong:

```
if my_boolean == True:  
    do_something()
```

Worse:

```
if my_boolean is True:  
    do_something()
```

Still bad:

```
if len(my_list) != 0:  
    do_something()
```

Correct for sequences and booleans

```
if some_variable:  
    do_something()
```

- ▶ Always use a `def` statement instead of an assignment statement for anonymous (lambda) expressions

Wrong

```
f = lambda x: 2 * x
```

Correct

```
def double(x):  
    return 2 * x
```

Good Documentation Should...

- ▶ Explain what each function / module / package does or is responsible for
- ▶ Be understandable to you when you revisit the code in 6 months
- ▶ Be understandable by someone new to the project (but not *necessarily* new to the subject matter)
- ▶ Be specific and to the point

- ▶ Start with a single line docstring
- ▶ Include additional documentation as necessary
- ▶ Always document arguments and returns

```
def complex(real=0.0, imag=0.0):
```

```
    """Form a complex number.
```

```
    Here is where you would put some additional, in-depth documentation.
```

```
    Keyword arguments:
```

```
        real -- the real part (default 0.0)
```

```
        imag -- the imaginary part (default 0.0)
```

```
    Returns:
```

```
        An imaginary number corresponding to the given arguments
```

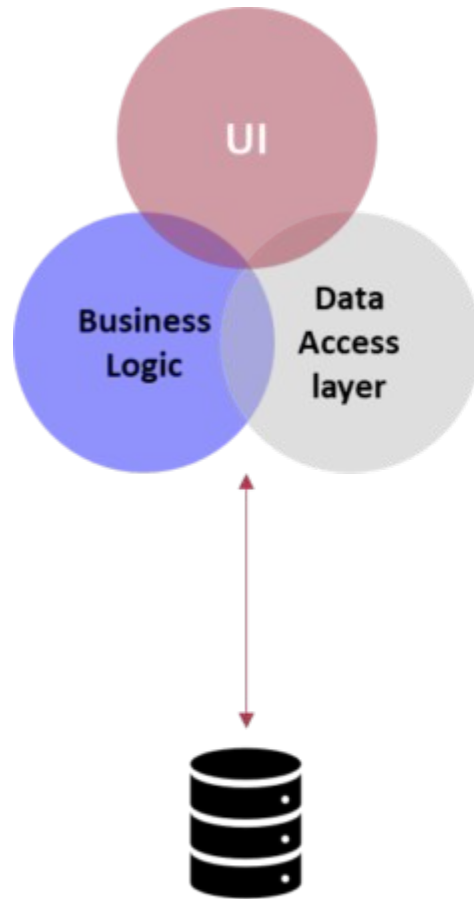
```
    """
```

- ▶ Triple quotes are **always** used
- ▶ The closing quotes are on the same line as the opening quotes.
- ▶ The docstring is a phrase ending in a period. It prescribes the function's effect as a command ("Do this", "Return that"), not as a description; e.g. don't write "Returns the pathname ."

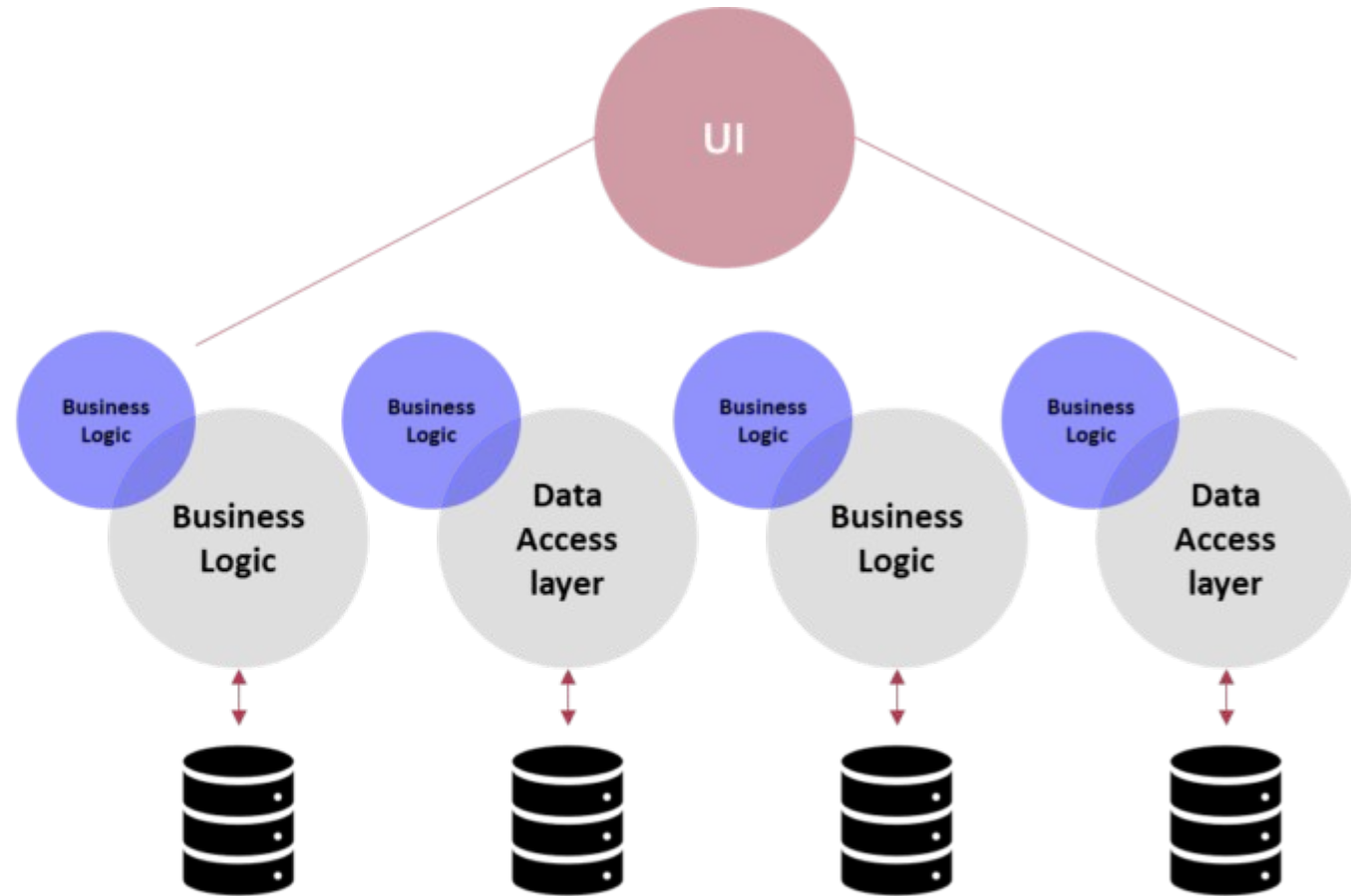
```
def kos_root():  
    """Return the pathname of the KOS root directory."""  
  
    ...
```

Can find the official documentation for PEP at the following link:

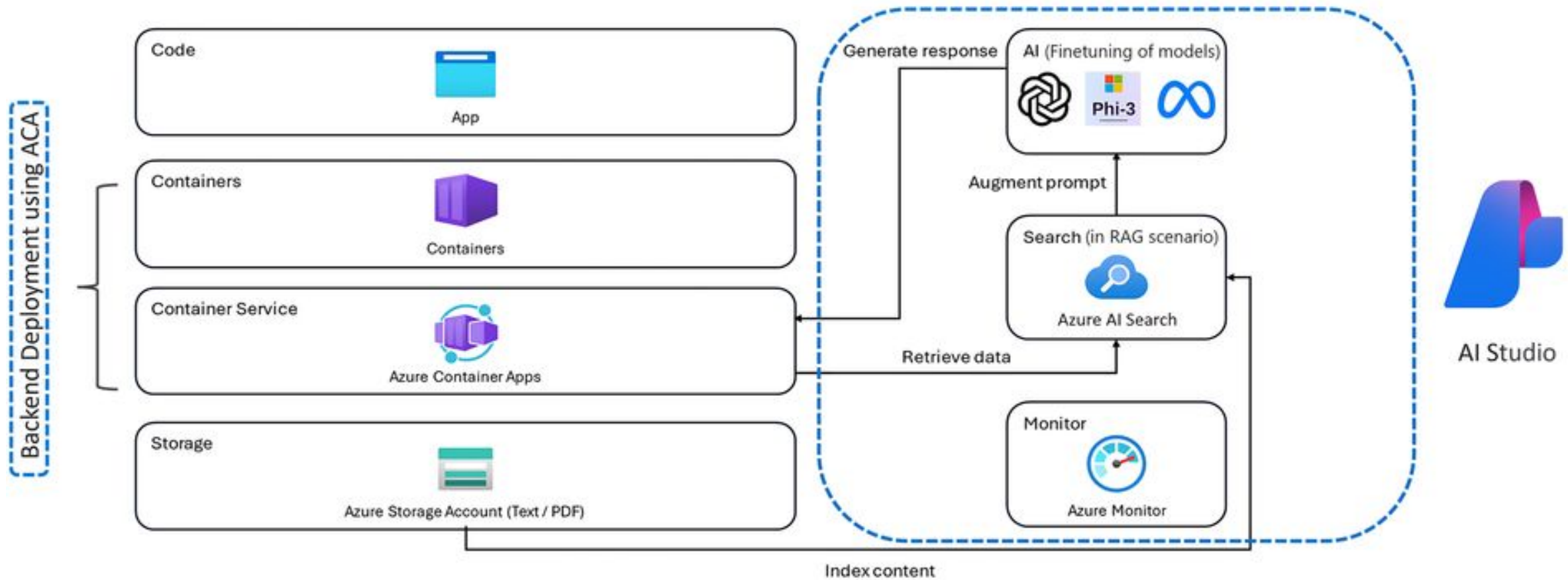
<https://peps.python.org/>



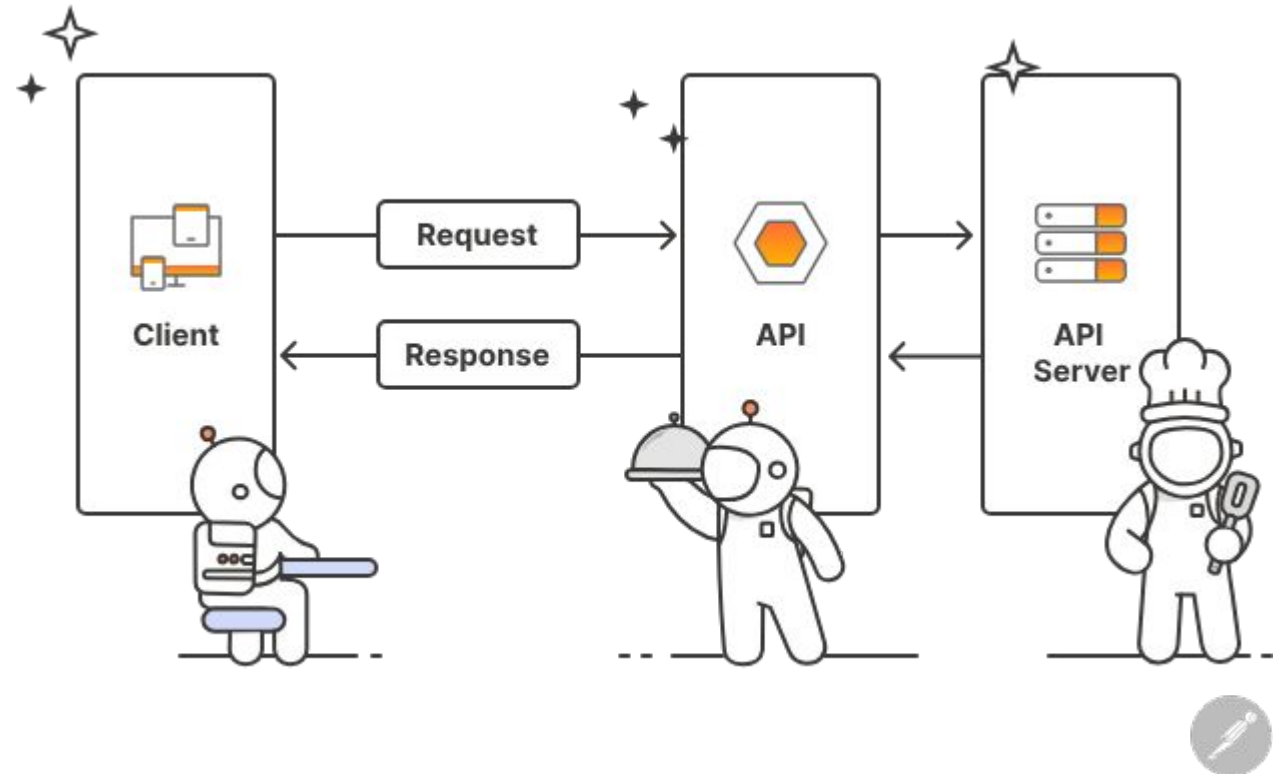
**MONOLITHIC
ARCHITECTURE**



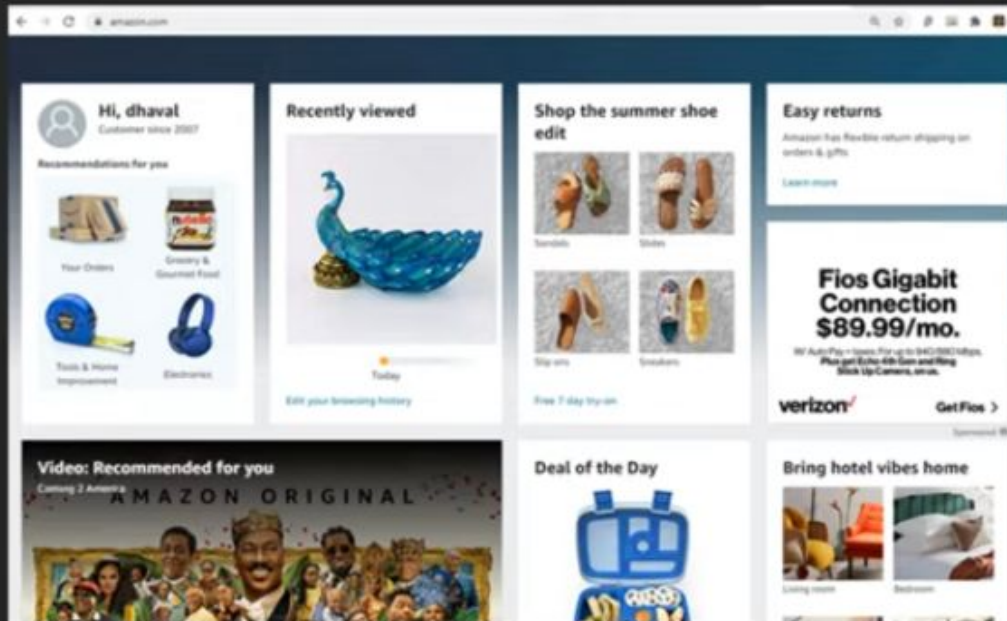
**MICROSERVICE
ARCHITECTURE**



An application programming interface (API) is a set of programming codes that queries data, parse responses, and sends instructions between one software platform and another.



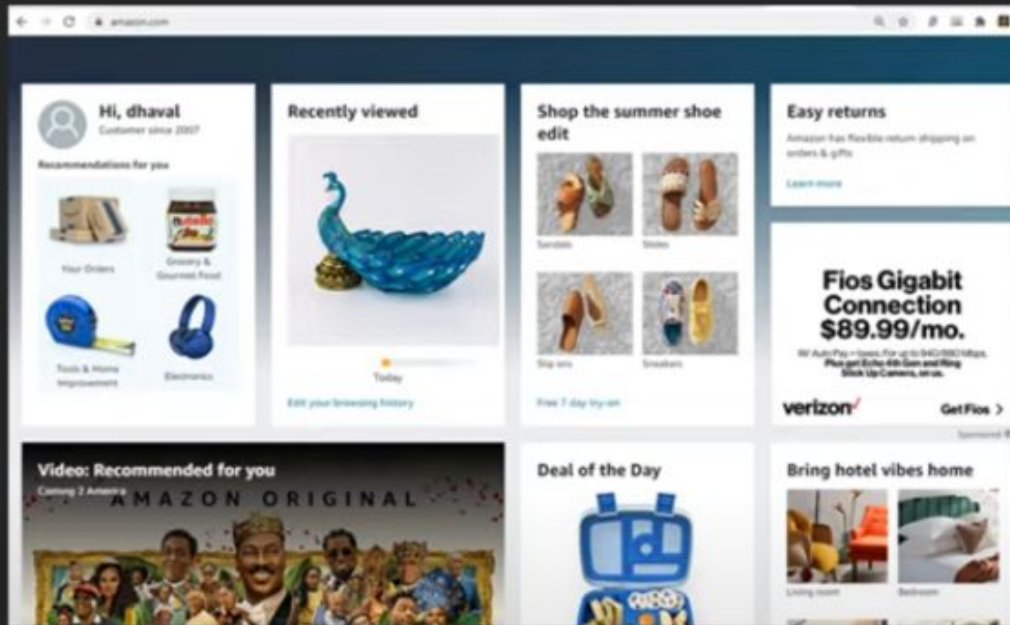
UI



Backend

amazon.com/place_order

UI



Backend

amazon.com/place_order

FastAPI

GET

Read data

Show me
iPhone
covers

POST

Create data

Create new
order

PUT

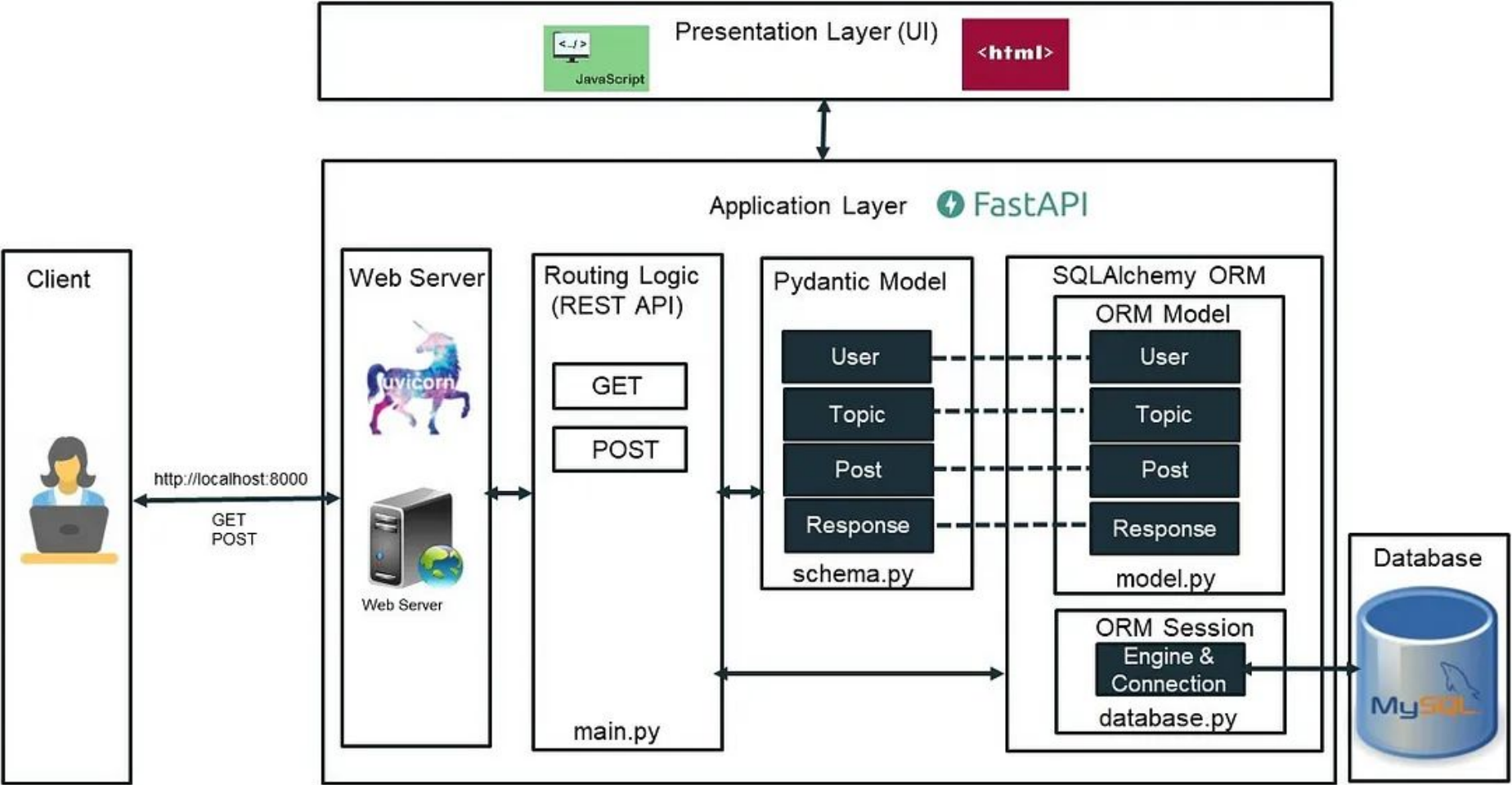
Update data

Update an
order

DELETE

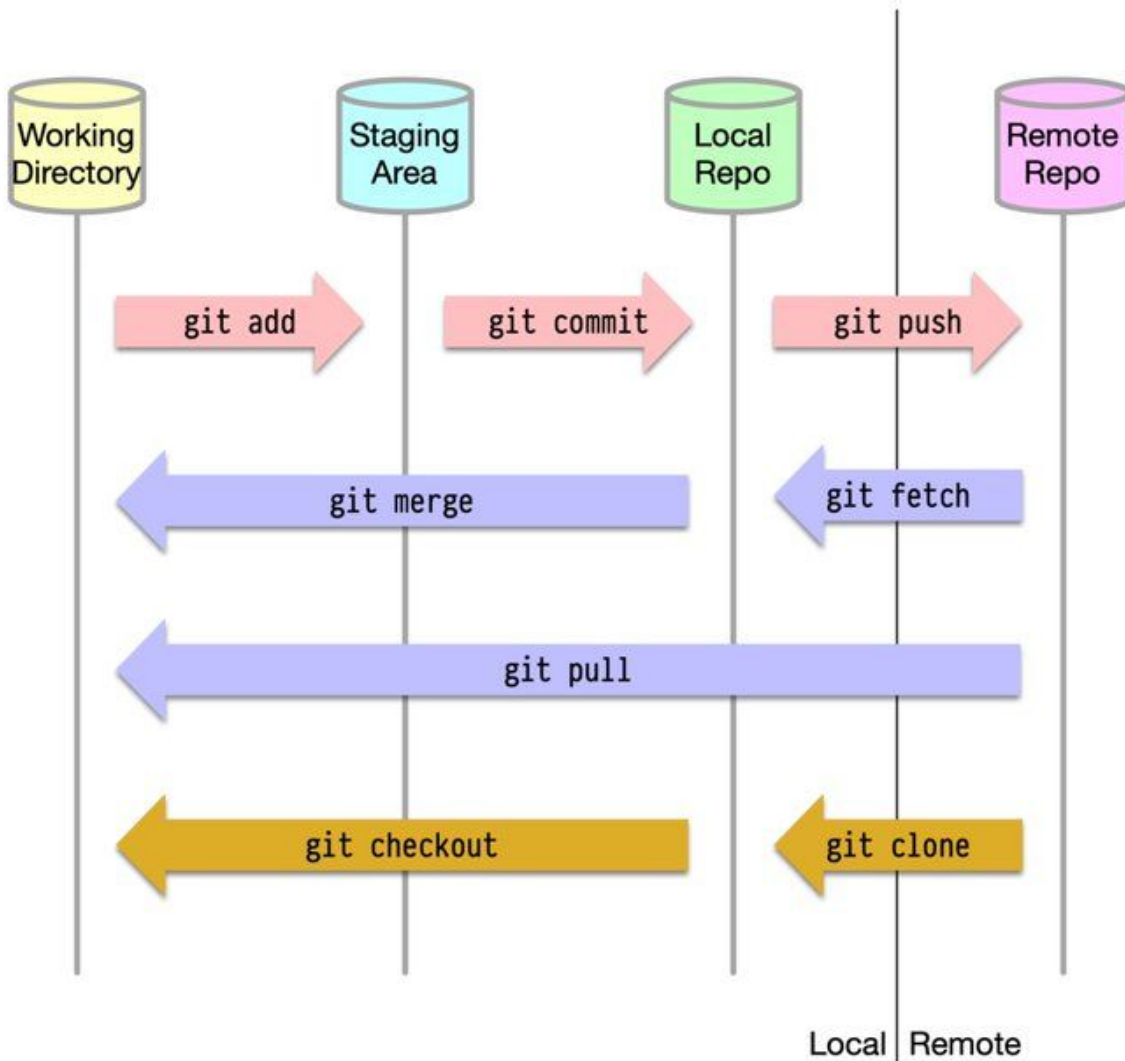
Delete data

Delete an
order

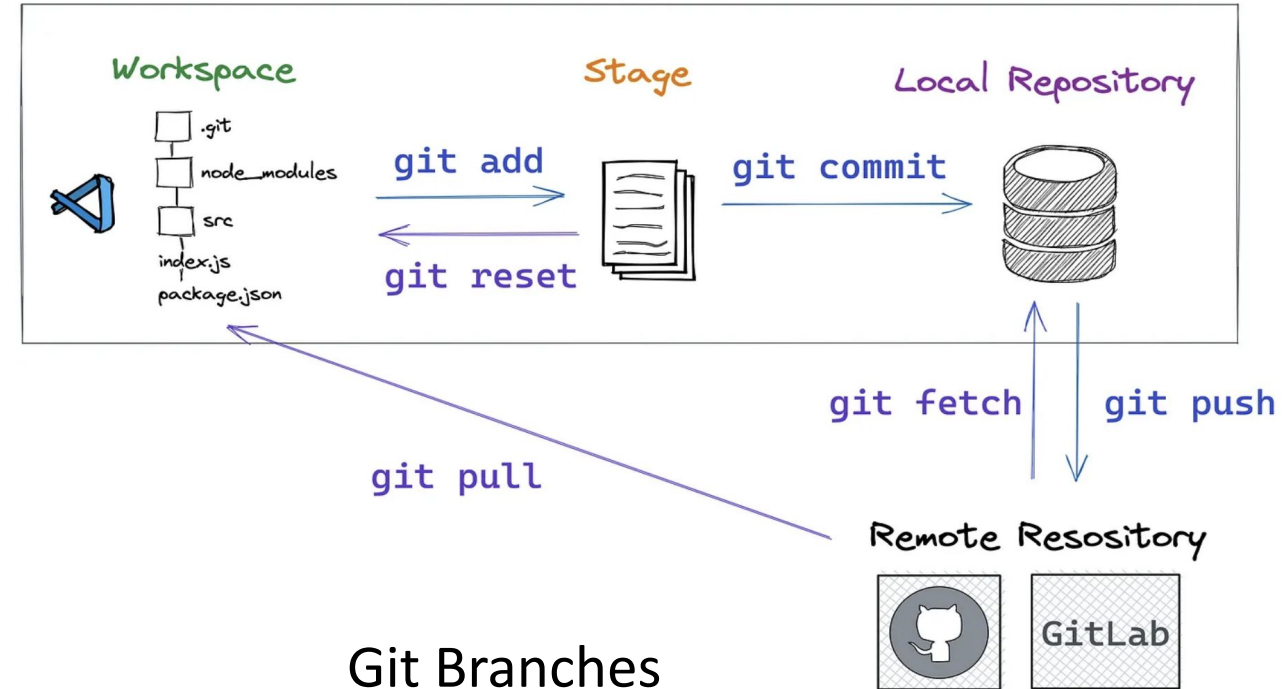


How Git Commands work

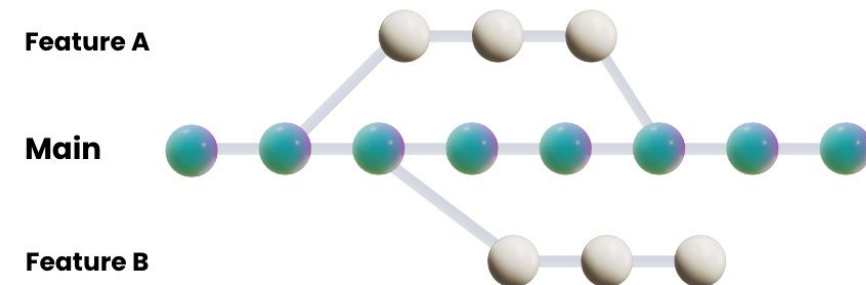
ByteByteGo.com



Local



Git Branches



INSTALLATION & GUIs

With platform specific installers for Git, GitHub also provides the ease of staying up-to-date with the latest releases of the command line tool while providing a graphical user interface for day-to-day interaction, review, and repository synchronization.

GitHub for Windows

<https://windows.github.com>

GitHub for Mac

<https://mac.github.com>

For Linux and Solaris platforms, the latest release is available on the official Git web site.

Git for All Platforms

<http://git-scm.com>

SETUP

Configuring user information used across all local repositories

```
git config --global user.name "[firstname lastname]"
```

set a name that is identifiable for credit when review version history

```
git config --global user.email "[valid-email]"
```

set an email address that will be associated with each history marker

```
git config --global color.ui auto
```

set automatic command line coloring for Git for easy reviewing

SETUP & INIT

Configuring user information, initializing and cloning repositories

git init

initialize an existing directory as a Git repository

git clone [url]

retrieve an entire repository from a hosted location via URL

STAGE & SNAPSHOT

Working with snapshots and the Git staging area

git status

show modified files in working directory, staged for your next commit

git add [file]

add a file as it looks now to your next commit (stage)

git reset [file]

unstage a file while retaining the changes in working directory

git diff

diff of what is changed but not staged

git diff --staged

diff of what is staged but not yet committed

git commit -m "[descriptive message]"

commit your staged content as a new commit snapshot

BRANCH & MERGE

Isolating work in branches, changing context, and integrating changes

git branch

list your branches. a * will appear next to the currently active branch

git branch [branch-name]

create a new branch at the current commit

git checkout

switch to another branch and check it out into your working directory

git merge [branch]

merge the specified branch's history into the current one

git log

show all commits in the current branch's history

INSPECT & COMPARE

Examining logs, diffs and object information

git log

show the commit history for the currently active branch

git log branchB..branchA

show the commits on branchA that are not on branchB

git log --follow [file]

show the commits that changed file, even across renames

git diff branchB...branchA

show the diff of what is in branchA that is not in branchB

git show [SHA]

show any object in Git in human-readable format

TRACKING PATH CHANGES

Versioning file removes and path changes

```
git rm [file]
```

delete the file from project and stage the removal for commit

```
git mv [existing-path] [new-path]
```

change an existing file path and stage the move

```
git log --stat -M
```

show all commit logs with indication of any paths that moved

IGNORING PATTERNS

Preventing unintentional staging or committing of files

```
logs/  
*.notes  
pattern*/
```

Save a file with desired patterns as .gitignore with either direct string matches or wildcard globs.

```
git config --global core.excludesfile [file]
```

system wide ignore pattern for all local repositories

SHARE & UPDATE

Retrieving updates from another repository and updating local repos

```
git remote add [alias] [url]
```

add a git URL as an alias

```
git fetch [alias]
```

fetch down all the branches from that Git remote

```
git merge [alias]/[branch]
```

merge a remote branch into your current branch to bring it up to date

```
git push [alias] [branch]
```

Transmit local branch commits to the remote repository branch

```
git pull
```

fetch and merge any commits from the tracking remote branch

REWRITE HISTORY

Rewriting branches, updating commits and clearing history

git rebase [branch]

apply any commits of current branch ahead of specified one

git reset --hard [commit]

clear staging area, rewrite working tree from specified commit

TEMPORARY COMMITS

Temporarily store modified, tracked files in order to change branches

git stash

Save modified and staged changes

git stash list

list stack-order of stashed file changes

git stash pop

write working from top of stash stack

git stash drop

discard the changes from top of stash stack


```
DhakaTrafficDetection/
├── data/
│   ├── traffic_data
│   └── traffic.yaml
├── saved_models/
│   ├── Best_Accuracy_Dhaka_AI_Yolov5L_By_Autobot_BS_8.pt
│   ├── Dhaka_AI_Yolov5S_With_BS_8.pt
│   └── Dhaka_AI_Yolov5S_With_BS_16.pt
├── src/
│   ├── data_preparation.py
│   ├── inference.py
│   ├── model_training.py
│   └── utils.py
├── templates/
│   └── index.html
├── uploads/
│   ├── .gitkeep
│   └── images.jpeg
├── .gitignore
├── Dockerfile
├── main.py
├── README.md
├── requirments.txt
├── setup_kaggle.sh
└── streamlit_app.py
```

Follow Code Modular Code in Github:

<https://github.com/AlFahim/DhakaTrafficDetection/>